Microsoft Azure

Path-Based Routing

Complete Setup Guide for Beginners

From Zero to Working Application Gateway

What You Will Learn

By the end of this guide, you will have set up a working Azure Application Gateway that routes traffic to two different App Services based on URL path.

- Create and deploy two Flask apps (Frontend + API) on Azure App Services
- Connect GitHub to Azure for automatic deployments
- Configure an Application Gateway with Path-Based Routing
- Set up Health Probes, Backend Pools, and Routing Rules
- Troubleshoot common 502 errors

Final Architecture (What We Are Building)

When everything is done, this is how traffic will flow:

User visits	http://<your-public-ip>/
Goes to	Frontend App Service (Flask UI)
User visits	http://<your-public-ip>/prasad
Goes to	Backend API App Service (Flask API)
Traffic router	Azure Application Gateway

Note: *The Application Gateway acts like a traffic cop — it reads the URL and decides which App Service to send the request to.*

Phase 1: Create Your Two Flask Apps

1.1 What Is a Flask App?

Flask is a lightweight Python framework to build web applications. We need two apps:

- Frontend App — shows a webpage when you visit /
- Backend API App — returns data when you visit /prasad

1.2 Create Frontend App (app.py)

Create a file called app.py with this content:

```
from flask import Flask

app = Flask(__name__)
@app.route('/')
def home(): return 'Hello from Frontend!'
app.run()
```

1.3 Create Backend API App

Create a SEPARATE folder for the API. Same structure but different route:

```
@app.route('/prasad')

def api(): return 'Hello from API!'
```

Note: *Keep these two apps in separate GitHub repositories. Each app gets its own App Service.*

Phase 2: Deploy Apps to Azure App Service

2.1 Create App Service for Frontend

Follow these steps in the Azure Portal (portal.azure.com):

Step 1: Search App Services

In Azure Portal, search 'App Services' in the top search bar and click it.

Step 2: Click + Create

Click the blue '+ Create' button on the top left.

Step 3: Fill in Basic Settings

Subscription	Select your Azure subscription
Resource Group	Create new or use existing (e.g., myResourceGroup)
Name	frontend-app (must be globally unique)
Runtime stack	Python 3.11
Region	East US (or any region close to you)
Pricing Plan	Free F1 (for learning/testing)

Step 4: Click Review + Create

Review your settings and click 'Create'. Wait ~2 minutes.

2.2 Repeat for Backend API App

Repeat all steps above but change the Name to: api-app (or similar unique name).

> **Note:** *You will now have TWO App Services running — one for frontend, one for API.*

Phase 3: Connect GitHub to Azure (Deployment Center)

3.1 Why Use Deployment Center?

Deployment Center connects your GitHub repo to Azure App Service. Every time you push code to GitHub, Azure automatically updates your app. No manual uploads needed.

3.2 Setup for Frontend App

Step 1: Open your Frontend App Service

Go to App Services > click your frontend app.

Step 2: Find Deployment Center

In the left sidebar, scroll down and click 'Deployment Center'.

Step 3: Choose GitHub as Source

Under 'Source', select 'GitHub' from the dropdown.

Step 4: Authorize GitHub

Click 'Authorize' and log in to your GitHub account.

Step 5: Select Repository

Choose your Organization, Repository (frontend repo), and Branch (main).

Step 6: Save

Click 'Save' at the top. Azure will now auto-deploy on every push.

Note: *After saving, go to the 'Logs' tab in Deployment Center to watch the first deployment happen in real time.*

3.3 Repeat for Backend API App

Do the same steps for your API App Service, but select your API GitHub repository.

3.4 Test Your Apps

After deployment, test both apps directly before setting up the gateway:

- Frontend URL: https://frontend-app.azurewebsites.net/ — should show 'Hello from Frontend!'
- API URL: https://api-app.azurewebsites.net/prasad — should show 'Hello from API!'

Note: *If either app does not work here, fix it BEFORE proceeding to Application Gateway setup.*

Phase 4: Create the Application Gateway

4.1 What Is an Application Gateway?

An Application Gateway is a Layer 7 (HTTP/HTTPS) load balancer. It can route traffic to different backend servers based on:

- URL path (e.g., / goes to one server, /prasad goes to another)
- Domain name (host-based routing)
- Custom rules

Think of it as a smart traffic cop sitting between the internet and your apps.

4.2 Create the Application Gateway

Step 1: Search

In Azure Portal, search 'Application Gateways' and click it.

Step 2: Click + Create

Click '+ Create' button.

Basics Tab:

Resource Group	Same as your App Services
Name	my-app-gateway
Region	Same region as your App Services
Tier	Standard V2
Autoscaling	Yes (min: 0, max: 2)
Availability Zone	None (for testing)

Frontends Tab:

Frontend IP type	Public
Public IP address	Create new — name it: gateway-public-ip

Backends Tab (Backend Pools):

Click 'Add a backend pool'. Create TWO pools:

Pool Name	Target Type	Target Value
frontend-pool	App Service	frontend-app.azurewebsites.net
api-pool	App Service	api-app.azurewebsites.net

Phase 5: Configure Health Probes

5.1 What Is a Health Probe?

A Health Probe is a regular check that Application Gateway sends to your backend apps. It asks: 'Are you alive?' If an app does not respond, the gateway stops sending traffic to it.

5.2 Create Health Probes

After the gateway is created, go to your Application Gateway > Health Probes > + Add.

Health Probe for Frontend:

Name	frontend-probe
Protocol	HTTP
Host	frontend-app.azurewebsites.net
Path	/

Interval	30 seconds
Timeout	30 seconds
Unhealthy threshold	3

Health Probe for API:

Name	api-probe
Protocol	HTTP
Host	api-app.azurewebsites.net
Path	/prasad
Interval	30 seconds
Timeout	30 seconds
Unhealthy threshold	3

> **Note:** *Always click 'Test' before saving a health probe. It should return Status 200 OK.*

Phase 6: Configure Backend Settings

6.1 What Are Backend Settings?

Backend Settings tell the Application Gateway HOW to talk to your backend apps — what protocol to use (HTTP/HTTPS), what port, and importantly whether to override the host header.

6.2 Why Host Header Override Matters

Azure App Services only respond to requests with the correct host header. When Application Gateway forwards a request, it must send the right host name. Without this, you get 502 errors.

6.3 Create Backend Settings

Go to your Application Gateway > Backend Settings > + Add. Create two settings:

Settings for Frontend:

Name	frontend-settings
Backend protocol	HTTPS
Backend port	443

Override with new host name	Yes
Host name override	Pick from backend target — select frontend-app
Cookie based affinity	Disabled
Custom probe	frontend-probe

Settings for API:

Name	api-settings
Backend protocol	HTTPS
Backend port	443
Override with new host name	Yes { path as [/] }
Host name override	Pick from backend target — select api-app
Cookie based affinity	Disabled
Custom probe	api-probe

Phase 7: Set Up Path-Based Routing Rules

7.1 What Are Routing Rules?

Routing Rules are the core of Path-Based Routing. They define: IF the URL path matches X, THEN send traffic to backend pool Y.

7.2 Create the Routing Rule

Step 1: Go to Rules

In your Application Gateway, click 'Rules' in the left sidebar.

Step 2: Add a Rule

Click '+ Add routing rule'.

Step 3: Fill in Listener

Rule name	path-routing-rule
Priority	100
Listener name	http-listener
Frontend IP	Public

Protocol	HTTPS
Port	443

Step 4: Configure Backend Targets

Click the 'Backend targets' tab.

Target type	Backend pool
Backend target (default)	frontend-pool
Backend settings (default)	frontend-settings

Step 5: Add Path Rule for API

Click '+ Add multiple targets to create a path-based rule'.

Path	/prasad*
Target name	api-path-rule
Backend target	api-pool
Backend settings	api-settings

> **Note:** *The * after /prasad means it matches /prasad and anything after it like /prasad/data. Use /prasad* not just /prasad.*

Step 6: Save

Click 'Add' then 'Save'. Wait 2-3 minutes for changes to apply.

Phase 8: Test Your Setup

8.1 Find Your Public IP

Go to Application Gateway > Overview. Copy the 'Frontend public IP address'.

8.2 Test Both Routes

Test 1 — Frontend	Open browser: http://<your-public-ip>/
Expected result	Shows Your Frontend
Test 2 — API	Open browser: http://<your-public-ip>/prasad
Expected result	Shows Your API Page

If both work — Congratulations! Your Path-Based Routing is fully working.

Phase 9: Troubleshooting — 502 Bad Gateway

9.1 What Is a 502 Error?

A 502 error means Application Gateway received the request but could not get a valid response from the backend App Service. Most common causes:

- Host header not set correctly in Backend Settings
- Health Probe is failing (app not running)
- Wrong backend pool target (typo in app service URL)
- App Service is stopped or crashed

9.2 How to Fix

Step 1: Check Backend Health

Go to Application Gateway > Backend Health. All pools should show 'Healthy' in green.

Step 2: Check Host Header Override

Go to Backend Settings. Make sure 'Override with new hostname' is set to YES and the correct app is selected.

Step 3: Test Health Probe

Go to Health Probes > click your probe > Test. It should return 200 OK.

Step 4: Test App Service Directly

Visit your App Service URL directly (e.g., https://frontend-app.azurewebsites.net). If this fails, the problem is with the app itself, not the gateway.

Step 5: Check App Service is Running

Go to App Service > Overview. Status should show 'Running'. If it shows Stopped, click Start.

What You Completed

- **Azure App Services — created and deployed two Flask apps**
- **GitHub Deployment Center — connected for auto-deploy**
- **Application Gateway — created with public IP**
- **Backend Pools — one for frontend, one for API**

- **Backend Settings — with host header override**
- **Health Probes — monitoring both apps**
- **Path-Based Routing — / to frontend, /prasad to API**
- **502 Troubleshooting — know how to debug**